Multi-Bottleneck Fairness in RDMA Fabrics: Measurement and an Explicit-Rate Service Mode

Haoyu Wang
University of Massachusetts Boston
Boston, MA, USA
haoyu.wang001@umb.edu

Bo Sheng
University of Massachusetts Boston
Boston, MA, USA
bo.sheng@umb.edu

Zerin Shaima Meem
University of Nebraska Omaha
Omaha, NE, USA
zmeem@unomaha.edu

Xiaoqian Zhang
University of Nebraska Omaha
Omaha, NE, USA
xiaoqianzhang@unomaha.edu

Abstract

Deployable RDMA congestion control is evaluated almost exclusively on single-bottleneck benchmarks, while datacenter traffic increasingly crosses several contended links in series. We measure multi-bottleneck fairness on lossless RDMA fabrics and find that every scheme we evaluate — DCQCN, TIMELY, DCTCP, HPCC, PowerTCP — is unfair to the multi-bottleneck flow (1.6–2.3× against a fluid max-min reference), HPCC's instance a stable near-winner-take-all collapse. Rate traces identify an η-hold state in HPCC that freezes startup asymmetry; a controlled experiment shows why sender-side reconstruction of an explicit fair rate cannot repair it: independently initialized rate recursions preserve history, while one shared per-link register erases it. Carrying that register's value in a single 64-bit INT field (RDMA-RCP, three words of state per port) restores the max-min *relative* allocation (1.005×, zero PFC) and cuts a ring-collective coflow's completion time by 18–23%. We equally measure the boundaries: mixed HPCC/RCP traffic requires telemetry isolation, and high fan-in requires a lengthened control interval. Stock HPCC's control logic and wire format are unchanged; its regression outputs are byte-identical to the pre-RDMA-RCP baseline.

1 Introduction

HPCC [20] and its derivatives represent the state of the art in datacenter RDMA congestion control: the switch stamps raw per-hop state into INT headers, and the sender computes a precise utilization estimate toward a target η, matching or beating DCQCN, TIMELY, and DCTCP on the canonical single-bottleneck benchmarks — which is where deployable RDMA congestion control is, almost exclusively, evaluated.

Modern datacenter traffic, however, increasingly crosses *several contended links in series*: cross-pod ML collectives [27], disaggregated memory and storage, multi-tenant aggregation and core layers. The practical stake is concrete: a collective completes when its *slowest* flow completes; in the collectives we study, cross-fabric multi-bottleneck flows frequently determine completion. On a 16-flow ring-collective coflow over a k=4 fat-tree, we measure stock HPCC inflating collective communication completion time by ∼27% over what max-min allocation delivers — at every ECMP hash seed we test (§5.4). Whether deployable RDMA congestion control is fair to such traffic has not been carefully measured. This paper measures it.

Max-min fairness is a policy choice, not a universal congestion-control objective; our claims target traffic classes whose completion is gated by multi-bottleneck flows, and §5.14 delimits where it is the wrong target. We proceed with a deterministic, oracle-grounded methodology. Our simulator is the original HPCC authors' ns-3 code base, upgraded to a tagged modern release (ns-3.45): the changes are interface-level — build-system, header, and deprecated-API adaptations — while the congestion-control logic, switch datapath, and wire formats are the original code (one latent field-initialization bug, exposed only by the 376-node topology, is fixed and documented). We validated the upgrade by reproducing the original paper's baseline comparisons across five schemes and two production workloads (§5); the reproduction record and raw results are public. The measurements here are therefore produced by the original HPCC implementation's logic, not a re-implementation. Our findings:

(1) **The failure spans every scheme we evaluate, and we characterize HPCC's instance** (§2, §5.3). Every evaluated controller is unfair to the multi-bottleneck flow — DCQCN 1.70–2.32×, TIMELY 1.83–2.01×, DCTCP 1.61–1.78×, HPCC 1.90–1.96×, PowerTCP 1.79–1.86× at $N = 2$–4 — all growing with bottleneck count, and robust to start-time jitter. For HPCC we isolate the mechanism: the failure is *near-winner-take-all* (the long flow collapses to ∼0.4 Gbps against ∼23 Gbps competitors; 3.5× for small flows), robust to the tested RTT variations, immune to HPCC's multi-rate mode and PINT compression, and compounded past four bottlenecks by the `maxHop` = 5 wire cap corrupting the control input. A single-bottleneck control (N=1: 1.10–1.39× pairwise spread, direction set by arrival order) separates HPCC's baseline rate lottery from the systematic multi-bottleneck penalty on top.

(2) **Private replication of the fair-rate computation cannot close the gap — shared placement can** (§3). We evaluate six sender-only designs: five heuristics, plus the most direct variant — a full sender-side mirror of the RCP recursion driven by the same INT-visible inputs the switch would use. The mirror still fails (1.56× even with synchronized starts; 1.65×/0.81× under staggered arrival, the *direction* set by arrival order), while the identical law at the switch holds ≈1.0×; per-flow fair queueing equalizes only what senders offer — windowed HPCC leaves 1.97×, windowless

HPCC equalizes at ~13× the oracle FCT. The mechanism is history: independently maintained rate state preserves arrival-order asymmetry indefinitely, while a shared per-link register eliminates it — evidence about explicit-rate state placement, not a formal impossibility (§3).

(3) **An RCP service mode over INT restores max-min, verified end to end** (§4, §5). RDMA-RCP carries a switch-computed RCP fair rate in one 64-bit INT field with three words of mutable state per egress port, alongside stock HPCC (logic, wire format, and regression outputs unchanged). It reaches penalty ≈1.005× across bottleneck counts, sizes, start patterns, and ECMP seeds — rate-only or canonically windowed — cutting ring-coflow JCT by 18–23% (18% under a byte-identical INT header). We equally report its deployment boundary: sharing a link with stock HPCC serves neither class, even under a weighted-DRR reservation, because both controllers read port-global telemetry (§4.3) — a fabric-wide or isolated-service mode. The mechanism is RCP; we claim the measurement study and the placement result, not a new control law.

Artifacts. The implementation, the topology/workload generators, the max-min oracle, and deterministic scripts that regenerate the figures (`make_figures.py`) and the ablation/sensitivity/baseline tables (`run_sensitivity.py`) from source are available in our artifact-review repository.[1]

2 Background and Motivation

2.1 HPCC, briefly

HPCC's per-hop utilization estimate for hop i is

$$u_i = \frac{txRate_i}{C_i} + \frac{\min(qlen_i, qlen_i^{\text{prev}}) \cdot maxRate}{C_i \cdot W} \tag{1}$$

where the first term reflects bandwidth utilization and the second is a queue-drain term that keeps standing queues bounded. In single-rate mode the sender takes $U = \max_i u_i$, smooths it over the base RTT, and applies $curRate/\max c + R_{AI}$ when $\max c = U/\eta \geq 1$, else $curRate + R_{AI}$. The window W is sized to $m_{win} \cdot rate/NIC_line$ under `var_win`.

2.2 The multi-bottleneck parking lot

The *parking lot* is a classic congestion-control fairness benchmark [7, 17]. Because every link is equally contended, the *max-min fair* allocation [7] gives every flow an equal share, so the long flow should run exactly as fast as each cross flow despite traversing many contention points rather than one. The parking lot therefore isolates the multi-bottleneck fairness question in its simplest form, with no confounding asymmetry in link capacity or hop count of the competitors. One control is essential before reading any deviation as a multi-bottleneck effect: on a *single* shared bottleneck (N=1) with two symmetric 50 MB flows, stock HPCC already allocates unequally (37.20 vs 26.72 ms, a 1.39× spread), while RDMA-RCP equalizes them exactly (33.89 vs 33.89 ms). Stock HPCC therefore carries a baseline rate unfairness between symmetric flows; the multi-bottleneck penalty we characterize is the *additional* deviation on top of it, growing to 1.90–1.96× at N = 2–4.

We construct a parameterized parking lot with N inter-switch bottlenecks (25 Gbps) and 100 Gbps host links (Figure 1): one "long" flow crosses all N, one "cross" flow crosses each; 50 MB each, simultaneous start, so the max-min rate is 12.5 Gbps for every flow. All experiments are deterministic (byte-identical across repeated runs).

Metric. We compare each flow's actual flow completion time (FCT) against an overhead-free **fluid max-min reference** ("oracle" hereafter): the ideal FCT under the max-min allocation *policy*, computed on *payload* bandwidth with no propagation, packetization, header, queueing, or feedback cost. It is not a per-flow lower bound under arbitrary allocations: an unfair controller can overserve a flow, so $P_i < 1$ is possible. A solo calibration flow (no contention) completes 5.8% above the bound on wire and protocol overhead alone (`run_round5.py`), anchoring how much of a small P_i excess is accounting, not control. We compute the allocation by *progressive filling* [7]: all active flows rise in lockstep until a link saturates; those flows freeze; repeat until all are frozen. Because flows arrive and depart over time, we run it *event-driven* — recomputing progressive filling at every flow arrival and completion — so the reference tracks the active set exactly; it is exact for our fluid topologies. The headline metric is the **unfairness ratio** $\max_{\text{path}} FCT/\text{mean}(\text{short-path FCT})$; the oracle's value is 1.0 for equal-size, simultaneous flows.

2.3 Findings F1–F4: characterizing the gap

Magnitude. Stock HPCC's unfairness ratio at $N \in \{2, 3, 4\}$ is 1.90, 1.92, 1.96: the multi-bottleneck flow's FCT is roughly twice that of single-bottleneck competitors. **There is a non-monotonic regime change at** $N \geq 5$, where the long flow becomes favored (0.51×); this turns out to be an INT-overflow artifact at `maxHop` = 5 and we exclude $N \geq 5$ from the rest of this paper's stock comparisons (Figure 2).[2]

Robustness. Size and start-time perturbations give 1.81–3.53× (Figure 4), worst (3.5×) when the multi-bottleneck flow is small — the latency-sensitive RPC case. *Multi-rate HPCC* (per-hop EWMA, $\min_i Rc_i$) helps but does not fix it (1.44–1.75×, growing with N). *RTT-equalization rules out RTT bias:* the penalty stays ~1.9× across a 4.5× sweep of cross-flow RTTs (cross-flow RTT up to 1.8× the long flow's).

Trace diagnosis. At N = 4 (Figure 5 left) the long flow collapses to ~0.38 Gbps while the four cross flows hold ~23.2 Gbps each for the entire 19 ms contention window, with zero PFC and zero loss — *near-winner-take-all*: at every shared link the local single-bottleneck flow wins and the long flow is squeezed. This is an equilibrium, not a transient: with 500 MB flows (10× the contention, ~7,000 RTTs) the ratio is 1.975×, the long flow flat at 0.39 Gbps for 150 ms. *Nor removed by tuning the principal rate-control knobs:* sweeping $\eta \in \{0.90, 0.95, 0.98\}$, R_{AI} to 25× its reference (1 Gb/s), and min-rate

[1] https://github.com/UNOCourseDemo/hpcc-fs

[2] The 5-hop cap is HPCC's original wire format, not an artifact of our ns-3 upgrade: the reference simulator hard-codes `maxHop` = 5 in its INT header [2], consistent with the paper's 42-byte worst-case INT overhead (five 8-byte hop records). Conversely, a parking-lot path at $N \leq 4$ traverses at most five switches — *within* the INT budget — so the retained N = 2–4 measurements are structurally free of the overflow.

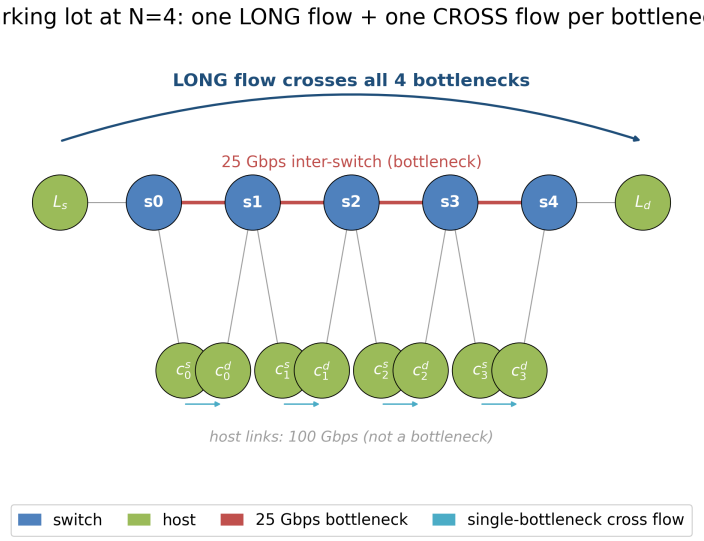


Figure 1: Parking lot at N=4: the long flow crosses every bottleneck, each cross flow exactly one. Host links are not bottlenecks.

at N=4 leaves the ratio at 1.74–1.97×, PFC 0 (`run_round5.py`); the largest R_{AI} helps most — additive recovery is directionally right but orders too slow (§3).

3 The Limits of Sender-Side Control

A natural first instinct — one we explored exhaustively — is to keep all state at the sender, preserving HPCC's stateless switch. Six sender-only variants sit behind a new `mb_mode` knob (`mb_mode = 0` byte-identical to stock, verified): five heuristic changes to `UpdateRateHp`, and the most direct — a full sender-side mirror of the RCP recursion.

3.1 Five heuristic variants

The five heuristics attack different levers: a k-aware additive boost (1.63×, best, at γ=50); a debiased max/mean blend (1.95× — per-hop u_i nearly equal); responsibility-weighted and k-th-root MD (1.89/1.92× — MD barely fires at η-hold); a share-aware AI on every flow (1.71×). Stock: 1.96×; target 1.0.

3.2 Isolating placement: identical observations, private vs. shared state

We isolate placement in a deterministic fluid model of one link: every controller receives identical (y, q) at identical instants — same rule, gains, initialization; only the *location of the recursion state* differs (`run_consistency.py`; flow 2 arrives at t=5 ms). Shared register: the late arrival inherits the evolved state, 12.5/12.5 Gbps. Private, synchronized: the copies remain identical — raw (y, q) information is not the deficit in this controlled case. Private, staggered: the newcomer initializes fresh, both multiply identically thereafter, and the ratio freezes at exactly 2:1. From an independently initialized local recursion, a sender cannot reconstruct the register's *evolved state*, even under identical subsequent (y, q) — transmitting that state is precisely what RDMA-RCP does.

3.3 The full stack: a virtual-RCP mirror

The fluid model isolates the mechanism; `mb_mode 6` confirms it in the full stack: the sender runs the switch's recursion *privately* per hop from its own INT samples of C, y, q (identical rule and gains, $R_0 = C/2$, its base RTT as interval), sending at $\min_i \hat{R}_i$. The real stack adds per-flow sampling and feedback-delay asymmetry beyond the fluid model's arrival asymmetry — why the mirror falls short even synchronized.

The mirror reaches only 1.560× synchronized, 1.654/0.813× with the long/cross side 5 ms late (direction set by arrival order); the identical law at the switch holds 1.003–1.008×.

3.4 The structural reason: consistency, not information

Two observations explain the pattern. *First, the η-hold equilibrium:* once the contended links settle at target utilization η, HPCC's multiplicative term $1/\max c \approx 1$ and every flow holds its current rate — whoever grabbed more at startup keeps it. Additive nudges (modes 1, 5) are orders of magnitude too slow relative to flow lifetimes (200 ms $\approx 10^4$ RTTs, no convergence); MD-side modifications (modes 3, 4) rarely fire at the hold point.

Second — what mode 6 isolates — private explicit-rate state preserves history. The recursion's raw inputs are sender-visible (exactly in the fluid model; sampled in the stack); what a private copy cannot reproduce is the register's evolved *history*. A multiplicative update on independently initialized state preserves the ratio set at arrival forever (§3.2); the same update on one shared per-link register makes every flow — incumbent or newcomer — adopt the same R within one RTT, eliminating the history dependence. This is why RCP [11, 13] and XCP [17] place the computation in the network. The scheduling side sharpens the picture (`run_round4.py`): per-flow equal-weight DRR at every port under unchanged (windowed) HPCC leaves 1.97× — the NIC-scaled window underfeeds

the long flow; a scheduler cannot allocate service never offered. Removing the window lets DRR equalize (0.997×) — zero fairness error, an extreme efficiency gap: ∼0.9 Gbps per flow, ∼13.6× every flow's oracle FCT: rates settle near the sender floor (the η-hold's additive path back is too slow) and the fabric idles; a fixed maxBDP window sits between (1.52×). Scheduling can enforce *equality*; the max-min *allocation* — equal shares at full utilization — also requires a signal telling each sender how much to offer, and the absolute oracle-relative metric separates the outcomes (13.6× vs 1.06×). Per-flow queues also mean per-flow state at every port; the register needs three words. The claim is scoped: for private reconstructions of an explicit fair rate, state placement is decisive — six sender designs fail, the identical law at the switch succeeds. Not a formal impossibility: contractive AIMD-style designs escape the history argument in principle, though the AIMD-flavored schemes measured (DCQCN, DCTCP) still show 1.6–2.3× (§5.3).

4 RDMA-RCP Design

§3 motivates a design that (a) makes *dominant* flows yield rather than nudging victims, and (b) places a fair-rate *signal* at the switch. No flow counting: each port runs a standard RCP law on aggregate load and queue whose fixed point *is* the max-min rate C/N. We package the most minimal such mechanism as an *operating mode* alongside stock HPCC on the same INT transport, selected per fabric or isolated slice (§4.3).

Background: RCP. The Rate Control Protocol (RCP) [11, 13] is a router-assisted congestion control in which each link advertises a *single* rate R — the rate *each* flow should send at, not an aggregate budget — to all flows crossing it, holding no per-flow state. Routers stamp R into passing packets and each flow sends at the minimum R along its path. With N flows each sending R, the controller drives $NR \to C$ (idle link $\Rightarrow$ R rises; standing queue $\Rightarrow$ R falls), so at equilibrium $R \approx C/N$ — the max-min (processor-sharing) share of the flows bottlenecked there, tracked *without counting* N: as flows arrive or leave, R self-adjusts. RCP supplies §3's missing ingredient: a fair-share signal computed where flow count is implicitly observable — the link. RDMA-RCP runs this law once per egress port and carries its R in the INT header; the path-minimum R approximates network-wide max-min fairness (ratio within 1.2% of the oracle's; absolute overheads in §5.7): a flow bottlenecked elsewhere contributes less load here, and the loop hands the slack to the rest.

4.1 The mechanism

The entire protocol is Algorithms 1 and 2: one scalar fair rate per egress port, path-minimum stamped into a single header field, adopted by the sender from the returning ACK — no per-flow switch state, no per-hop record, no receiver changes.

At the switch (Alg. 1). Each egress port keeps a single scalar fair rate R_j, refreshed once per control interval d — a fabric-wide constant set to the maximum base RTT (21.4 μs by default; §§5.9–5.11 vary it) — by the standard RCP rule $R \leftarrow R\,(1 + (\alpha(C - y) - \beta q/d)/C)$, where y is the load observed over the last interval and q the queue backlog. The term $\alpha(C - y)$ chases spare capacity (positive while the link is under-utilized, pushing R up) and $\beta q/d$ brakes when a standing queue forms (pulling R down); they balance

Algorithm 1 RDMA-RCP switch — per egress port j

State R_j (fair rate), t_j (last update), B_j (txByte snapshot)
Const C_j link capacity; α=0.4, β=0.226; d control interval (≈ RTT); R_{floor}
procedure OnDequeue(pkt, j) ▷ every packet leaving port j
  if R_j uninitialised **then**
    $R_j \leftarrow C_j/2$; $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$ ▷ conservative start
  end if
  if $now - t_j \geq d$ **then** ▷ one control interval elapsed
    $y \leftarrow (txBytes_j - B_j) \cdot 8 \,/\, (now - t_j)$ ▷ observed load
    $q \leftarrow queueBytes_j \cdot 8$ ▷ backlog (bits)
    $R_j \leftarrow R_j \cdot \big(1 + (\alpha(C_j - y) - \beta\, q/d)/C_j\big)$ ▷ RCP update
    $R_j \leftarrow \text{clamp}(R_j, R_{\text{floor}}, C_j)$
    $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$
  end if
  if $pkt.fairRate = 0$ **then** ▷ first hop on the path
    $pkt.fairRate \leftarrow R_j$
  else
    $pkt.fairRate \leftarrow \min(pkt.fairRate, R_j)$ ▷ path-min
  end if
  $txBytes_j \leftarrow txBytes_j + \text{sizeof}(pkt)$
end procedure

Algorithm 2 RDMA-RCP sender (receiver echoes the header unchanged)

on QP-create: $rate \leftarrow R_{\text{start}}$ ▷ cold-start; distinct from R_{floor}

procedure OnAck(a)
  if $a.fairRate = 0$ **then return** ▷ no signal yet
  end if
  $rate \leftarrow \text{clamp}(a.fairRate, R_{\text{floor}}, NICrate)$ ▷ adopt path-min fair share
end procedure

at $y \approx C$, $q \approx 0$, which is the $R \approx C/N$ fixed point above. We use the RCP defaults $\alpha = 0.4$, $\beta = 0.226$ (untuned; the result is insensitive to them, §5.9). The port's entire state is three numbers — R_j, the last update time, and a byte-counter snapshot — with *no* per-flow data.

On the wire. The FS INT mode (`IntHeader::FS`) carries one 64-bit field, `fairRate`; the serialized cost is 8 bytes per packet *independent of hop count* (vs. 8 bytes *per hop* for stock HPCC). Each switch min-aggregates R_j into this field (first hop initialises it); the receiver echoes the header unchanged.

At the sender (Alg. 2). The new ACK handler `HandleAckFs` reads `fairRate` from the returning ACK and sets the sending rate to it, clamped to $[R_{\text{floor}}, NICrate]$. That single assignment is the whole sender control loop. Two parameters are deliberately distinct: the cold-start rate R_{start} (1 Gbps; sent before the first ACK) and the adopted-rate floor R_{floor} (100 Mbps; `fs_min_rate`) — conflating them turns $N \leq C/R_{\text{start}}$ into a hard fan-in bound (measured in

§5.11). ACKs are per-QP, in order, on a fixed reverse path, so the newest write wins; reordering multipath would need an epoch.

4.2 Startup smoothing

Cold start: the switch initializes $R = C/2$ (not C, else the first stamp is line rate); the sender starts at $R_{\text{start}} = 1$ Gbps so the first RTT does not blast; adopted rates may fall to R_{floor} thereafter. The signal is path-complete in one RTT; convergence reaches 5% of fair share by ~0.6 ms, 1% by 2.3 ms (§5); signaling does not grow with hop count, and fairness is insensitive to the tested path-length/RTT variations.

4.3 Additive integration and coexistence

RDMA-RCP is `cc_mode == 11`. The mode-3 (HPCC) and mode-10 (PINT) wire formats and switch/sender code paths are untouched; we add a case to the INT header mode and serialization, the mode dispatches, and one branch in the validation driver. The driver also forces the QP window to 0 in FS mode (rate-only control; §5.8 shows why a NIC-scaled window must not remain). In the shipped `cc_mode 11` the INT wire format is a global choice — one mode per fabric. To test per-class deployment we added `cc_mode 12`: stock's 42-byte layout (stock packets unchanged) with per-packet class dispatch; the FS class pays stock's full header cost — conservative for our claim. Both gates hold (all-stock ≡ mode 3 exactly; all-FS ≡ mode 11's 1.005×, zero PFC).

Coexistence is not free (negative result). With the long flow in the RCP class and crosses in the stock class (N=4), *neither class gets its fair share*: rate-only RCP overshoots (21.6 vs 12.5 G fair; 0.51×), the windowed variant under-shoots (6.2 Gbps, 2.53×), and a separate priority queue does not help (HPCC's queue stays near zero; it forfeits its turns) — non-compliant flows are invisible to R's inference exactly as flow count is invisible to a sender.

We then tested the obvious remedy: a weighted-DRR egress scheduler (byte-deficit DRR, `dwrr_weights`) reserving each class its exact 50% per-link entitlement. *It changes nothing* (0.511×): the scheduler never binds, because stock HPCC's INT still reports *port-global* bytes and queue — the stock class sees $u \approx 1$ from RCP traffic, holds low, and never offers its reserved share, which RCP absorbs. The reservation must extend into the *telemetry* (per-class C, y, q — true slicing) or a separate fabric (`run_mixed_mode.py`; PFC zero throughout).

5 Evaluation

5.1 Methodology

All experiments use the deterministic parking-lot benchmark of §2, driven by our `hpcc-validation.cc`. Flows are 50 MB unless noted; sim time is 200 ms; the optimized binary is used throughout. We compare against stock HPCC (`cc_mode 3`), stock HPCC multi-rate, HPCC-PINT, and PowerTCP (`cc_mode 13`: law and constants verbatim from `inet-tub/ns3-datacenter` @ `4dd55d8`; single-bottleneck sanity matches stock HPCC, 828 vs 830 μs, equal peak queue — reducing the likelihood that the multi-bottleneck result is a porting artifact). The simulator is the original HPCC ns-3 code base upgraded to a tagged ns-3.45 release — interface-level changes only; control logic and wire formats unchanged — verified by reproducing the original paper's baseline comparisons (HPCC vs. DCQCN, TIMELY, DCTCP, and PINT across WebSearch and FB-Hadoop workloads at 30–70% load on a 320-server fat-tree). The full reproduction record ships with the artifact,[3] and its verdict is that the upgraded code matches the original paper's central findings. Metrics, used consistently below: *unfairness ratio* = long-flow FCT / mean cross-flow FCT (oracle = 1.0); *oracle-relative FCT* P_i = per-flow FCT / oracle FCT ($P_i < 1$: the flow took capacity the policy assigns to others); *generalized unfairness* (heterogeneous cases) = worst-long / best-flow P_i; *raw long/short ratio* = mean long / mean short FCT (ECMP study); *slowdown* = FCT / same-size standalone FCT; *coflow JCT* = max FCT over a coflow. We separate *fairness error* (deviation of relative allocations from the max-min structure; the unfairness ratios) from the *efficiency gap* (absolute distance from the fluid bound; P_i, utilization) — a controller can be fair yet inefficient, and several results below differ exactly there. "Zero PFC" statements refer to the evaluated buffer/PFC configuration (§5.10).

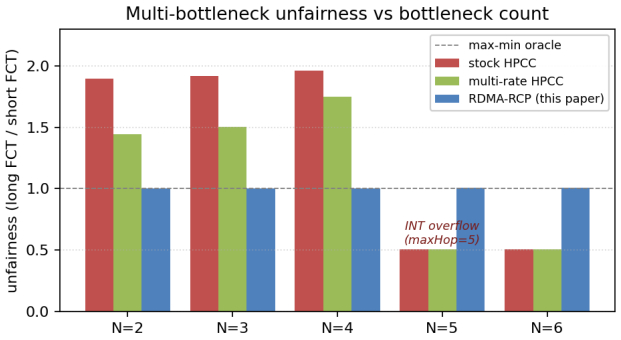


Figure 2: Unfairness vs bottleneck count N. RDMA-RCP is flat ≈1.0; stock and multi-rate HPCC fail at $N \leq 4$ and break at $N \geq 5$ (INT overflow).

5.2 Headline: N-sweep

Stock and multi-rate HPCC are starved across $N = 2$–4 (penalty 1.90–1.96× for stock, 1.44–1.75× for multi-rate), then break entirely at $N \geq 5$, where the per-hop INT record overflows the `maxHop` cap (the apparent 0.51× is the overflow artifact). RDMA-RCP is flat at 1.003–1.007× across the entire sweep — its one-field format does not grow with path length. HPCC-PINT, carrying one compressed utilization field, inherits load-based control and still suffers ~1.8× (1.77–1.88×) — the fix requires a fair-share signal, not a smaller INT footprint. All RDMA-RCP runs here have zero PFC.

5.3 The failure spans our evaluated stack

Is multi-bottleneck unfairness HPCC-specific? We ran every scheme in our stack on the same benchmark with harness parity (window-based schemes keep their window; DCQCN, TIMELY, and DCTCP run rate-only as configured in the HPCC reference stack — its "DCTCP" is that stack's rate-based DCTCP-inspired baseline, not a kernel TCP, kept for parity). Table 1: every scheme *in our evaluated*

[3] Record: `examples/hpcc/repro-verification` in the artifact repository. Raw results (554 MB): https://doi.org/10.5281/zenodo.23271137, mirrored as GitHub release `data-v1`.

Table 1: Unfairness ratio and PFC pauses on the parking lot: every *evaluated* scheme fails multi-bottleneck max-min; all worsen with N.

scheme	N=2	N=3	N=4	PFC
DCQCN	1.70×	1.90×	2.32×	6–34
TIMELY	1.83×	1.96×	2.01×	72–156
DCTCP	1.61×	1.72×	1.78×	256–280
HPCC	1.90×	1.92×	1.96×	0
PowerTCP	1.79×	1.85×	1.86×	0
RDMA-RCP	**1.003**×	**1.003**×	**1.005**×	0

stack is unfair to the multi-bottleneck flow and worsens with bottleneck count — DCQCN worst at N=4 (2.32×). HPCC is the norm here, not an outlier. The gap covers the post-HPCC generation: PowerTCP [1] (INT variant; port validated in §5) shows 1.79–1.86×, growing with N, PFC 0; its EWMA/AI softens the N=1 lottery (1.13 vs 1.39×) without approaching max-min (`run_round9.py`). Only the RCP mode reaches max-min. Untested controllers (Bolt, On-Ramp, Swift) are outside the claim.

5.4 When max-min is the objective: ring-collective coflow JCT

Collective communication motivates multi-bottleneck fairness: a step completes only when its *slowest* flow completes (coflow [9] JCT = max FCT); the slowest flows are the cross-pod, multi-bottleneck ones stock HPCC starves. We run a 16-flow ring coflow — one communication phase of a ring allreduce [26] (50 MB per host to its ring successor; 4 inter-pod, 4 intra-pod, 8 same-edge) over all hosts of the k=4 fat-tree, across five ECMP seeds. We also run the *full* collective — all $2(N-1) = 30$ phases of 3.125 MB chunks (decomposing a 50 MB tensor; the 50 MB single phase is a stress test), controller state carried — under *two* synchronization models: per-rank ring dependencies (each rank's next send gated on its own and its predecessor's completion — pipeline semantics at whole-chunk granularity; finer slice streaming unmodeled) and conservative global barriers. Pipelined: 44.1 → 34.8 ms, a 21.1% gain; barriered: 45.0 → 35.2 ms, 21.9% — robust to the synchronization model, since a slow ring edge throttles the pipeline's steady rate either way; phase JCTs $[1.17, 1.19]$ vs $[1.44, 1.56]$ ms; PFC 0; compute overlap unmodeled. The pipelined gain holds at 17.0% under the stock 42-byte header and across ECMP placements (20.2–21.6%, three seeds; `run_round10/11.py`). RDMA-RCP cuts JCT by 19.5–22.7% (mean 21.4%) at every seed, nearly placement-invariant (17.9–18.0 vs stock's 22.3–23.3 ms). Eight added 2 MB background flows leave the gain intact (+20.6%) and themselves speed up ∼3× (0.9–1.3 vs 2.2–4.3 ms) — a short flow's fate under near-winner-take-all is a placement lottery; max-min removes it. A header-size control: padding the RCP class to stock's 42-byte layout (§4.3, all-FS) still gives 19.0 vs 23.3 ms — 18% under byte-identical headers (23% at 8 bytes). The gain is fairness, not header thinness.

5.5 Structurally different topologies

Tree fabric (3-tier, no ECMP).. A 3-tier tree with unique shortest paths (15 nodes; 25 Gbps inter-switch, 100 Gbps host; Figure 3a). Stock penalty 1.36×, RDMA-RCP **1.003**×, PFC = 0.

k = 4 fat-tree (with ECMP).. The canonical fat-tree (20 switches + 16 hosts; Figure 3b), hash-based ECMP; 4 inter-pod (5-switch) plus 4 intra-pod (3-switch) flows. On the *oracle-normalized penalty* (long- vs short-path slowdown, each against the oracle, removing placement): stock 1.34×, RDMA-RCP **1.001**×. The *raw* long/short ratio stays ∼1.32× under RDMA-RCP— not CC unfairness but placement: ECMP hashes some flows onto under-loaded cores, inflating it for any controller. PFC 0 for both; path-min aggregation is path-invariant — ECMP needs no design changes. *Hash-seed sensitivity.* Because a single deterministic run fixes one placement, we re-ran the workload across *twenty* ECMP hash seeds (an additive `ecmp_seed_offset` knob shifts every switch's hash seed; the default preserves stock behavior), reporting the mean-long/mean-short raw ratio. Under stock HPCC that ratio varies 0.96–1.40× across seeds (mean 1.14) *with no change to the congestion controller* — direct evidence that the raw spread is placement-dominated. Under RDMA-RCP the mean is 1.04 and the median 1.001: fifteen of twenty seeds sit within 3% of parity; the five residuals (1.16–1.33×) are hash collisions putting long flows on shared cores, where equal FCTs are not the oracle either. At the default seed the contended case drops 1.338× → 1.001× (seed-robust; `run_ecmp_seeds.py`).

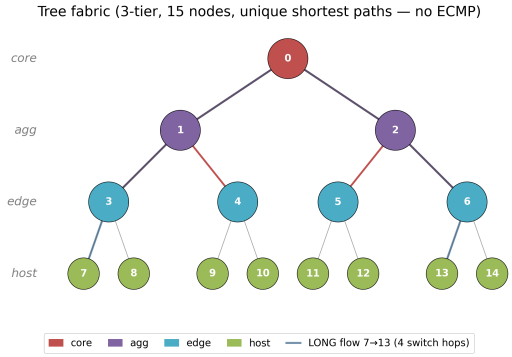
5.6 Robustness across asymmetric workloads

Across six perturbations (Figure 4) RDMA-RCP holds within 1.0 ± 0.012; the worst stock case (3.53×, small long flow) drops to 1.012×.

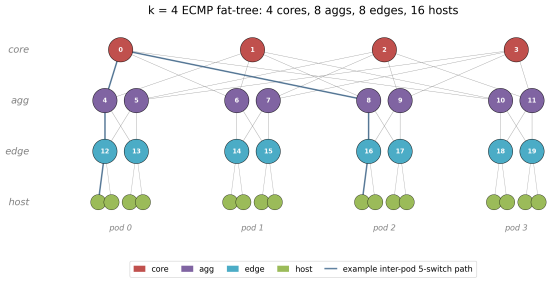
Start-time jitter. Jittering all five starts by $U[0, 1]$ ms (five seeds): stock 1.96–1.99×, RDMA-RCP 1.008–1.017×. On the N=1 control the pairwise spread persists at 1.10–1.39× across 10 μs–5 ms offsets, direction flipping with arrival order — the η-hold lottery, not synchronization — while RDMA-RCP equalizes at every offset.

5.7 Heterogeneous and dynamic contention

The uniform one-competitor parking lot is deliberately minimal, so we stress its assumptions (Table 2; `run_stress_matrix.py`): heterogeneous capacities, unequal competitor counts, two overlapping multi-bottleneck flows, and cross-flow churn, each scored against the event-driven oracle on the actual topology and schedule. A ratio alone can mask a uniform miss, so the table reports both *generalized unfairness* (worst-long / best-flow P_i; reduces to the headline ratio when uniform) and the absolute P_i range; the fully symmetric $\max_i P_i / \min_i P_i$ tells the same story (1.00–1.07 RDMA-RCP, 1.81–2.90 stock). Stock HPCC's unfairness grows with contention richness — worst with two overlapping longs (2.87×) and churn (2.23×), P_i scattered 0.41–1.34 — while RDMA-RCP holds 1.001–1.007× with $P_i \in [1.05, 1.13]$: *every* flow lands within 5–13% of its own oracle FCT — the 5.8% solo-flow calibration suggests a substantial fraction of the static-case excess is non-control overhead (the calibration is not path-matched; churn's 1.13 worst case carries real dynamics). Mean bottleneck utilization is *higher* than stock throughout (0.65–0.78 vs 0.51–0.70). PFC = 0 everywhere; 4–8× RTT spreads applied to the mode itself leave the ratio at 1.003–1.008×. Joint stress (churn



(a) Tree fabric — unique shortest paths, no ECMP.



(b) $k = 4$ ECMP fat-tree (4 pods).

Figure 3: Topology families beyond the parking lot. (a) 3-tier tree, 15 nodes; highlighted: the long flow $7 \rightarrow 13$ (four switches). (b) k=4 fat-tree (4 cores, 8 aggs, 8 edges, 16 hosts); highlighted: an inter-pod 5-switch path.

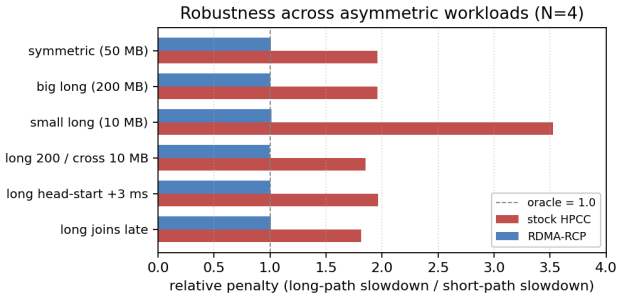


Figure 4: Asymmetric-size and staggered-start variants at N=4: RDMA-RCP within 1.0 ± 0.012 in all six; stock reaches 3.53× (small-long).

Table 2: Beyond the uniform parking lot: generalized unfairness and absolute P_i range. RDMA-RCP: ≈1.0×, every flow within 5–13% of oracle; PFC 0.

	stock HPCC		RDMA-RCP	
scenario	unf.	P_i range	unf.	P_i range
hetero cap. [25,50,25,100] G	1.82×	0.64–1.16	**1.003×**	1.06–1.06
unequal comp. [3,1,2,1]	1.41×	0.84–1.18	**1.001×**	1.06–1.06
two overlapping longs	2.87×	0.41–1.19	**1.007×**	1.05–1.06
cross-flow churn	2.23×	0.60–1.34	**1.002×**	1.06–1.13

+ 4× RTT spread + $d/4$ at once) leaves the ratio at 1.000× while absolute P_i rise to 1.21–1.39 — fairness error ≈0, efficiency gap 21–39%; the absolute metric exposes what the ratio hides (`run_round3.py`). (INT admits only rates {25, 50, 100, 200, 400} Gbps; heterogeneity stays within that set.)

5.8 Ablation: windows must scale with the fair rate

RDMA-RCP defaults to rate-only operation — the per-flow window cap is disabled (§4). Re-enabling HPCC's `var_win` degrades the penalty to 1.50/2.00/2.49× at $N = 2/3/4$, worsening with path length — that window is sized from the NIC line rate, not the multi-hop bottleneck. The problem is the window's *scale*, not windowing: $cwnd = R \cdot$ baseRTT (`fs_rcp_window`) matches rate-only fairness (1.004–1.006× vs 1.003–1.005×, zero PFC) while bounding in-flight data — a cap must track R, not the NIC.

5.9 Parameter sensitivity

We use the standard RCP defaults ($\alpha = 0.4$, $\beta = 0.226$) untouched. Sweeping one parameter at a time at $N = 4$ ($\alpha \in [0.2, 0.6]$, $\beta \in [0.1, 0.4]$, $R_0/C \in [0.25, 1]$, minRate $\in [100, 2000]$ Mb/s), the penalty stays within $[1.004, 1.008]\times$ with zero PFC throughout (even $R_0 = C$: the cold-start absorbs the first interval); the parameters are not load-bearing in this family (§5.7 adds a joint stress). The interval d tolerates misconfiguration asymmetrically: 2–4× too large or 2× too small stays 1.003–1.011×; 4× too small degrades to 1.212× (updates outrun feedback) — still far below every baseline, PFC zero.

5.10 Queueing and PFC

For context, every run uses the validated HPCC buffer model: 4 MB shared buffer with dynamic PFC thresholds, ECN marking $k_{\min}/k_{\max}/p_{\max} = 100\,\text{KB}/400\,\text{KB}/0.2$ at 25 Gbps (400/1600 KB at 100 Gbps), 1000 B payload, 1–2 μs link delays. With the smoothed startup (§4.2), RDMA-RCP produces **zero PFC events** on every workload in the N-sweep and the robustness matrix. ECN marking remains; queue length is bounded (peak ≈46 KB at $N = 4$, versus ≈118 KB for stock HPCC on the same workload); no flow loss. Do-no-harm: the RDMA-RCP additions (implementation name `HPCC-FS`, `cc_mode 11`) only add code; stock `cc_mode 3` smoke tests and N-sweep outputs are byte-identical to the pre-change baseline.

5.11 High fan-in and idle-port staleness

Incast fan-in, and the rate floor (`run_round4/5/6.py`): S senders to one 25 Gbps receiver link. Fan-in exposes why R_{start} and R_{floor} must not be conflated: with the floor tied to the 1 Gbps cold-start (our initial implementation), $S \le C/R_{\text{floor}} = 25$ is a *hard feasibility bound* — at S=64 senders are clamped above the 0.39 Gbps fair share and PFC enforces what they cannot adopt (2,045 pauses, 142 ms cumulative). Decoupling the floor (100 Mbps) removes the overload: PFC = 0 through S=128. What remains is a genuine *throughput* limitation, and it is not a startup transient: sweeping flow size at S=64, the mode's lifetime utilization (bound/FCT) is 0.47 at 200 KB and *falls* to 0.25 at 50 MB, versus stock HPCC's 0.92 at every size (64 pauses, 36.5 ms paused) — the port oscillates between multiplicative overshoot and the floor rail, and a lower floor deepens the collapse (0.26 at S=128, 50 Mbps floor). The decoupled floor also *moves* rather than removes the bound: $S \le C/R_{\text{floor}} = 250$ at 100 Mbps; at S=256 senders pin at the floor (util 0.99 only because floor ≈ fair share); beyond, structural overload. A receiver-side rate trace confirms the oscillation: goodput swings 3–25 Gbps, 91% of bins below $C/2$ (`run_round6.py`) — a pure efficiency gap, shares staying equal. Notably, naive loop gain does *not* explain the N-dependence — in the normalized no-delay recurrence fan-in cancels ($x' = x[1 + \alpha(1 - x)]$, $x = NR/C$; derivative $1 - \alpha$) — so it must enter through what that model omits: feedback delay, dequeue-triggered sampling, and the floor rail whose distance to the fair share shrinks with N (a lower floor deepens the collapse); a delay-aware analysis in the style of RCP's stability treatment [12] is future work. One fixed alternative operating point suffices: at $d = 2 \times RTT_{\max}$ (`fs_d_scale`) *every* key workload holds — headline N=4 1.003×, small-long unchanged, churn 1.007× with the same absolute P_i, ring coflow 18.0 vs 17.99 ms, fan-in utilization 0.84/0.70 at S=64/128 (both flow sizes at 64), all PFC-free; ×4 reaches 0.87–0.94 but takes 44 pauses at S=64, and a gap to stock's 0.92 remains (`run_round8.py`). $d = 2{\times}RTT_{\max}$ is thus a robust fixed compromise across the evaluated range: every fairness workload preserved, the fan-in efficiency gap much reduced but *not* eliminated (0.70 at S=128). The switch deliberately tracks no flow count, so the mode has a *provisioned* operating envelope, not an autonomously chosen timescale; scaling d with fan-in is future work. The floor never binds when fair shares exceed it; every fairness result reproduces exactly. *Idle-port staleness:* R updates only on dequeue, so an idle port freezes its last R. Congest a port with four flows, idle it $g \in \{0.1, 1, 10, 100\}$ ms, release a 1 MB probe: FCT 0.373 ms at *every* gap — faster than a fresh port's 0.713 ms (R_0=$C/2$ ramp): the drain tail's last dequeues restore R to ≈C — benign after *naturally draining* flows, the tested trajectory; an abrupt halt (failure, reroute) could freeze a low R (unmodeled; idle aging would cover it).

5.12 HPCC's home turf: lower queue, tunable FCT

What does the RCP mode cost on *single*-bottleneck workloads — HPCC's home turf? We measure the validated incast benchmark (3 senders → 1 receiver over 25 Gbps; 12 flows, 100–400 KB), sweeping the sender's startup rate. Two things stand out: RDMA-RCP holds far less queue at every startup (38–64 vs 143 KB), and HPCC's only lead — mean FCT at the 1 Gbps default (297.5 vs 258.6 μs, ∼15%) — closes and reverses at 8 Gbps (242.0 μs, one-quarter the queue, zero PFC, fairness headline undisturbed). The default thus trades ∼15% FCT for the queue win; aggressive startup is precisely what grows risky with fan-in (§5.11), so the headline runs keep 1 Gbps. Before the first fair-rate ACK a sender emits at most *startup* × baseRTT (≈2.7 KB at 1 Gbps); the $cwnd = R{\cdot}\text{RTT}$ variant (§5.8) restores a hard cap thereafter. RDMA-RCP still lacks HPCC's per-hop, window-bounded reaction (§5.11 measures the boundary); a hybrid capping HPCC at $\min(rate_{\text{HPCC}}, R)$ with a fair-rate-scaled window could recover it (future work).

5.13 Per-flow rate trace, before vs after

The before-vs-after rate trace (Figure 5) is the cleanest qualitative result: collapse to ∼0.4 Gbps under stock, lock-in at fair share under RDMA-RCP.

5.14 Scope of the headline claim

The ≈1.0× headline is a statement about the *congestion-control component* of multi-bottleneck unfairness on contention benchmarks — not a claim of universal FCT improvement on production workloads. Six first-class limits. (i) *Mixed short-flow workloads* do not exhibit the penalty we target: under WebSearch, stock's oracle-normalized penalty is already 1.001× (RDMA-RCP: 0.581× — short flows run *ahead* of their max-min share: policy deviation, not superiority), and on mean slowdown RDMA-RCP trades ∼15% off long flows for ∼45% on short paths. Short-flow-FCT workloads [4, 24] may prefer stock HPCC; max-min matters when multi-bottleneck flows gate a job (§5.4). A composite training-like mix run entirely in one mode (ring coflow + 15-to-1 parameter incast + 64 KB RPCs; `run_round7.py`): the job-gating coflow improves 9%; non-gating short classes stay sub-millisecond but slower (0.92/0.25 vs 0.64/0.05 ms; 15 transient pauses; the coflow finishes last in both modes, so whole-job completion = coflow completion) — the mode fits services gated by the collective. Per-service selection needs per-class link or telemetry isolation (§4.3: an unsliced port serves neither class); the deployment unit is a fabric or slice dedicated to max-min traffic, not a global replacement. (ii) *Fabric scale*: our $k = 6, 8$ scaling workload does not form persistent inter-pod contention (both penalties <1), so we add a *saturating* k=6 case (108 × 20 MB inter-pod flows, every stage at capacity; `run_round4.py`): coflow makespan 42.9 → 36.7 ms (−14%; −11% with the INT header padded to stock size), Jain index [15] 0.910 → 0.924, PFC = 0 at 45 switches. Both schemes' routes are identical (same hash, seeds, 5-tuples), so the makespan difference is caused by the controllers and their interaction with the same placement (2.9× vs 3.4× the 12.8 ms perfect-balance bound) — a scale-and-performance sanity check, *not* a max-min validation; a path-resolved oracle needs path instrumentation (k=8: future work). (iii) *ECMP placement*: a raw ∼1.3× long/short spread from hash placement remains; RDMA-RCP removes the CC component only. (iv) *Coexistence*: RDMA-RCP is a fabric-wide mode — unsliced sharing fails even under DRR (§4.3). (v) *High fan-in*: at $S \ge 64$ the rate mode sustains only 25–50% utilization at the default control interval (0.84–0.94 with a 2–4× interval, §5.11), and $S \le C/R_{\text{floor}}$ is a hard bound; with (iv), fan-in-heavy services favor the stock side of the slice. (vi) *Fairness unit*: max-min is per QP/flow — a

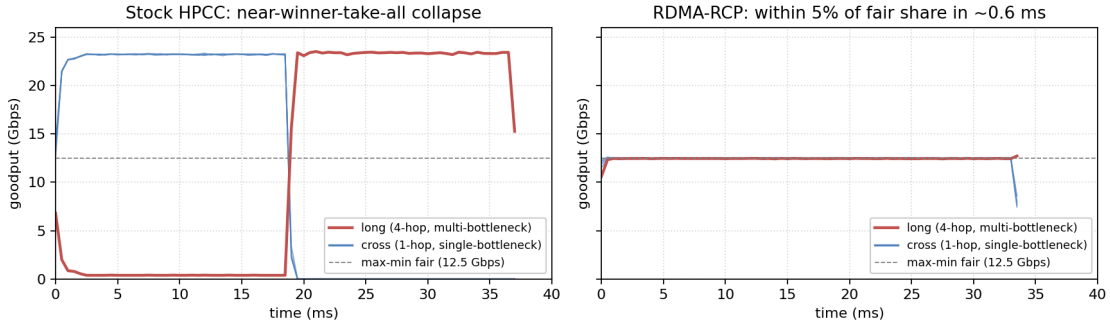


Figure 5: Per-flow goodput at N=4: stock (left) collapses the long flow to ~0.4 Gbps for 18 ms (near-winner-take-all); RDMA-RCP (right) converges all five to within 5% of fair share in ~0.6 ms (1% by 2.3 ms).

job opening two QPs measures exactly 2.00× a one-QP job's bandwidth (`run_round5.py`); job-level fairness would need weighted rates ($R_i = w_i R$) or QP policing (unevaluated).

6 Related Work

Datacenter transport. RDMA runs at scale over lossless Ethernet [14]; IRN [23] revisits losslessness itself. DCTCP [3], DCQCN [31], TIMELY [22], and Swift [19] trade buffering for latency via ECN- and delay-based control; Homa [24] and pFabric [4] prioritize short flows; HPCC [20] drives control from per-hop INT, with On-Ramp [21] and PowerTCP [1] extending the precise-signal philosophy. Across this line the target is single-bottleneck FCT, tail latency, or queueing; §5.3 shows every member we evaluate fails multi-bottleneck fairness.

In-network telemetry. INT [18] lets the data plane stamp per-hop state into packets on programmable pipelines, and HPCC was among the first to drive congestion control directly from it. The cost is header overhead, which later work minimizes: HPCC-PINT [6] compresses the per-hop record into one probabilistic field; Poseidon [30] restructures INT control for commodity deployability. RDMA-RCP continues this minimal-telemetry direction but changes *what* the field carries: an explicit fair *rate*, min-aggregated along the path — one 64-bit field, like PINT, yet a decision rather than a measurement.

Explicit-rate and switch-assisted control. Switch-computed rates are classic: ATM ABR's ERICA [16] computed per-port fair shares decades ago; XCP [17] split efficiency from fairness; RCP [11] reduced it to one packet-stamped fair-rate scalar per link, senders taking the path minimum. RCP is the closest mechanism to ours: RDMA-RCP adopts its update rule but situates it as a *minimal fairness signal injected into an existing INT load-signaling system* rather than a standalone protocol. Prior RCP work established explicit-rate fairness in general fabrics; unmeasured was whether deployable INT-based RDMA controllers exhibit the multi-bottleneck gap, and whether a telemetry-carried fair rate closes it in-stack — this paper's contribution. RDMA-RCP's per-port update and path-min aggregation *are* RCP's; the endpoint differs (rate-only by default; canonical window equal, §5.8), the interval is a fabric constant, updates event-driven, the port applies $R_0 = C/2$ and a floor. Over deploying RCP separately, RDMA-RCP adds packaging, not control: one field on the running INT transport, an unchanged stock path, per-mode selection — no second stack.

In-switch fairness. Fair queueing [10], DRR [28], CSFQ [29], and AFD [25] enforce fairness in the scheduler; §3 measures the boundary — scheduling equalizes only what senders offer, and without a rate signal the equal outcome is far from the max-min allocation. RDMA-RCP instead *advertises* one fair rate per port — no per-flow tracking or extra queues — on the match-action pipelines [8] INT assumes.

Closely related switch-assisted DC schemes. Bolt [5] accelerates HPCC's loop with sub-RTT per-flow assistance, targeting tail latency and ramp-up — a different objective: its feedback carries load/supply, not a fair share; its multi-bottleneck behavior is unknown (head-to-head: future work).

7 Discussion

Scaling and hardware. The saturating k=6 case is in §5.14; the non-saturating $k = 6, 8$ workloads form no persistent contention (penalties <1, 0 PFC, no added overhead). On hardware we offer a mapping, not a prototype: three registers per port (R, timestamp, byte snapshot); a dequeue-triggered update (two multiplies, two subtractions, a clamp); a 64-bit compare-and-write path-min per packet. Whether an atomic per-port stateful update can serialize the interval test and rate update within one target stage is *unverified*, as is dequeue-time queue-depth access. We took the step short of hardware: a *fixed-point simulation* (`fs_fixed_point`) — Q16 multiplier, precomputed $1/C$ and $\beta/(dC)$, a shift-plus-8-entry-mantissa reciprocal for the elapsed interval ($\leq$ 25% max error; idles to $2^{20}d$; idle probe 0.378 vs 0.373 ms), no divider — reproduces the N-sweep (1.001–1.004× vs 1.003–1.005×), ring JCT (+0.9%), and damped fan-in (0.81 vs 0.84; `run_round6.py`). One hazard surfaced: an R register in integer *Mbps* deadlocks floor recovery (growth truncates below ~2.5 Mbps; the fan-in case pins at the sender floor, 0.27) — Kbps units in the same register remove it. We therefore make 1 Kbps the *required* maximum granularity of R and the stamped field (32-bit unsigned Kbps spans 4.3 Tbps; the 64-bit wire field carries bps) — units, not width, were the binding choice. A Tofino prototype remains future work.

Limitations. Three topology families; in-sim baselines only (DCQCN, TIMELY, DCTCP, multi-rate HPCC, PINT, PowerTCP) — no Bolt, no testbed; the in-order per-QP ACK assumption (§4) holds in the evaluated stack but reordering multipath would need an epoch field; per-class coexistence requires link or telemetry isolation we do not design (§4.3); the ECMP study is a small seed-swept workload, not a realistic mix. All numbers come from one simulator — but its control logic is the original HPCC implementation with interface-level modernization only, verified by reproducing the original baselines; the $N \leq 4$ regime sits within the 5-hop INT budget by construction; a run on the unmodified legacy build remains future work.

8 Conclusion

Multi-bottleneck unfairness spans our evaluated stack, and HPCC's instance is a stable near-winner-take-all equilibrium, robust to the tested RTT variations (∼2×; 3.5× small flows; stable over 10× backlogs). Six sender-side corrections fail — including an exact private mirror of RCP — and per-flow fair queueing equalizes without approaching the allocation (1.97× windowed; ∼13× oracle FCT windowless): private explicit-rate state under identical observations freezes arrival-order asymmetry; a shared register erases it. RDMA-RCP carries that register's value in one INT field — three words of mutable state per port — reaching 1.005× — flows within 5–13% of oracle FCT in the heterogeneous matrix (21–39% under compound stress), PFC-free in the parking-lot, heterogeneous, coflow, and scale experiments — rate-only or canonically windowed, within 5% of fair share in ∼0.6 ms, hop-count-invariant signaling, stock HPCC's logic and outputs unchanged. Coexistence on a shared link fails even under DRR (§4.3); the mode belongs on sliced telemetry or a separate fabric.

References

[1] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. 2022. PowerTCP: Pushing the Performance Limits of Datacenter Networks. In *Proc. USENIX NSDI*. 51–70.

[2] Alibaba. 2019. HPCC reference simulator (ns-3). https://github.com/alibaba-edu/High-Precision-Congestion-Control. `int-header.h`: `static const uint32_t maxHop = 5`.

[3] Mohammad Alizadeh et al. 2010. Data Center TCP (DCTCP). In *Proc. ACM SIGCOMM*. 63–74.

[4] Mohammad Alizadeh et al. 2013. pFabric: Minimal Near-Optimal Datacenter Transport. In *Proc. ACM SIGCOMM*.

[5] Serhat Arslan, Yuliang Li, Gautam Kumar, and Nandita Dukkipati. 2023. Bolt: Sub-RTT Congestion Control for Ultra-Low Latency. In *Proc. USENIX NSDI*. 17 pages.

[6] Ran Ben Basat et al. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proc. ACM SIGCOMM*. 662–677.

[7] Dimitri Bertsekas and Robert Gallager. 1992. *Data Networks* (2nd ed.).

[8] Pat Bosshart et al. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014).

[9] Mosharaf Chowdhury and Ion Stoica. 2012. Coflow: A Networking Abstraction for Cluster Applications. In *Proc. ACM HotNets*.

[10] Alan Demers, Srinivasan Keshav, and Scott Shenker. 1989. Analysis and Simulation of a Fair Queueing Algorithm. In *Proc. ACM SIGCOMM*.

[11] Nandita Dukkipati. 2008. *Rate Control Protocol (RCP): Congestion Control to Make Flows Complete Quickly*. Ph. D. Dissertation. Stanford University.

[12] Nandita Dukkipati, Masayoshi Kobayashi, Rui Zhang-Shen, and Nick McKeown. 2005. Processor Sharing Flows in the Internet. In *Proc. IWQoS*.

[13] Nandita Dukkipati and Nick McKeown. 2006. Why Flow-Completion Time is the Right Metric for Congestion Control. *ACM SIGCOMM Comput. Commun. Rev.* 36, 1 (2006), 59–62.

[14] Chuanxiong Guo et al. 2016. RDMA over Commodity Ethernet at Scale. In *Proc. ACM SIGCOMM*.

[15] Raj Jain, Dah-Ming Chiu, and William Hawe. 1984. *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*. Technical Report DEC-TR-301. Digital Equipment Corporation.

[16] Shivkumar Kalyanaraman, Raj Jain, Sonia Fahmy, Rohit Goyal, and Bobby Vandalore. 2000. The ERICA Switch Algorithm for ABR Traffic Management in ATM Networks. *IEEE/ACM Transactions on Networking* 8, 1 (2000).

[17] Dina Katabi, Mark Handley, and Charlie Rohrs. 2002. Internet Congestion Control for Future High Bandwidth-Delay Product Environments. In *Proc. ACM SIGCOMM*. 89–102.

[18] Changhoon Kim et al. 2015. In-band Network Telemetry via Programmable Dataplanes. In *Proc. ACM SIGCOMM (Industrial Demo)*.

[19] Gautam Kumar et al. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proc. ACM SIGCOMM*.

[20] Yuliang Li et al. 2019. HPCC: High Precision Congestion Control. In *Proc. ACM SIGCOMM*. 44–58.

[21] Shiyu Liu et al. 2021. Breaking the Transience-Equilibrium Nexus: A New Approach to Datacenter Packet Transport. In *Proc. USENIX NSDI*.

[22] Radhika Mittal et al. 2015. TIMELY: RTT-based Congestion Control for the Datacenter. In *Proc. ACM SIGCOMM*. 537–550.

[23] Radhika Mittal et al. 2018. Revisiting Network Support for RDMA. In *Proc. ACM SIGCOMM*.

[24] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities. In *Proc. ACM SIGCOMM*.

[25] Rong Pan, Lee Breslau, Balaji Prabhakar, and Scott Shenker. 2003. Approximate Fairness through Differential Dropping. *ACM SIGCOMM Computer Communication Review* 33, 2 (2003).

[26] Pitch Patarasuk and Xin Yuan. 2009. Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. *J. Parallel and Distrib. Comput.* 69, 2 (2009).

[27] Alexander Sergeev and Mike Del Balso. 2018. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv:1802.05799* (2018).

[28] M. Shreedhar and George Varghese. 1995. Efficient Fair Queuing using Deficit Round Robin. In *Proc. ACM SIGCOMM*.

[29] Ion Stoica, Scott Shenker, and Hui Zhang. 1998. Core-Stateless Fair Queueing: Achieving Approximately Fair Bandwidth Allocations in High Speed Networks. In *Proc. ACM SIGCOMM*.

[30] Weitao Wang et al. 2023. Poseidon: Efficient, Robust, and Practical Datacenter CC via Deployable INT. In *Proc. USENIX NSDI*.

[31] Yibo Zhu et al. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proc. ACM SIGCOMM*. 523–536.